

Versi 2.0

Januari 2026



PRAKTIKUM SISTEM OPERASI

MODUL 6 - THREAD DAN MULTITHREAD

Pengarang:

Ir. Denar Regata Akbi, S.Kom., M.Kom.

Putri Nayla Sabri

Muhammad Zamzam Baihaqi

LABORATORIUM INFORMATIKA

Universitas Muhammadiyah Malang

DAFTAR ISI

DAFTAR ISI.....	2
PENDAHULUAN.....	2
TUJUAN.....	3
TARGET MODUL.....	3
PERSIAPAN.....	3
KATA KUNCI.....	3
LATAR BELAKANG TEORI.....	4
1. APA ITU THREAD DAN APA ITU MULTITHREADING?.....	4
1.1. Thread sebagai proses ringan.....	4
1.2. Program Single-threaded vs Multithreaded.....	5
1.3. Thread vs Proses: perbedaan utama.....	5
1.4. Mengapa menggunakan Multithreading? Manfaat dalam aplikasi nyata.....	6
2. MODEL THREAD DALAM SISTEM OPERASI.....	6
2.1. Thread Tingkat Pengguna (ULT).....	7
2.2. Thread Tingkat Kernel (KLT).....	7
2.3. Model Hibrida dan Implementasi Linux.....	8
3. MULTITHREADING DALAM PRAKTIK PADA ROCKY LINUX.....	9
3.1. Thread POSIX (pthreads): struktur dasar.....	10
LATIHAN 1.....	11
3.2. Membuat dan mengelola thread dengan 'pthread_create()'.....	11
LATIHAN 2.....	12
3.3. Mengamati thread dalam sistem ('ps -T', 'htop').....	12
LATIHAN 3.....	13
LATIHAN 4.....	13
4. MASALAH UMUM TERKAIT KONKURENSI DAN PRAKTIK AMAN.....	13
4.1. Kondisi balapan: ketika thread saling mengganggu.....	13
LATIHAN 5.....	14
4.2. Eksklusi bersama dasar dengan mutex ('pthread_mutex_t').....	15
4.3. Praktik terbaik: menghindari penghentian paksa dan kebuntuan.....	15
PRAKTIK.....	18
LATIHAN.....	18
RUBRIK PENILAIAN.....	21
SKALA PENILAIAN.....	21



PERKENALAN

TUJUAN

1. Memahami konsep thread sebagai unit eksekusi ringan dalam suatu proses.
2. Mengidentifikasi, membuat, dan mengamati thread menggunakan alat CLI seperti ps -T dan htop.
3. Terapkan teknik sinkronisasi dasar (mutex) untuk mencegah kondisi persaingan (race condition) dalam multithreading program.

TARGET MODUL

1. Bedakan antara program single-threaded dan multithreaded.
2. Kompilasi dan jalankan program POSIX Threads (pthreads) sederhana.
3. Pantau thread aktif dalam sistem menggunakan ps -T dan htop -H.
4. Terapkan pengecualian bersama dengan pthread_mutex_t untuk melindungi data bersama.

PERSIAPAN

1. Siswa Muslim diperbolehkan membaca doa berikut sebelum memulai pelajaran.

yy yyyyy yyy yyy yy yyy yy yyy yy · yyy yy yy yyy yy yyy yy yyy yy yyy yy y yyy y yyy yy yy yyy yy yyy yy

2. Laptop atau komputer pribadi 3.

Rocky Linux 4.

Virtual Box

5. Motivasi dan komitmen yang kuat untuk belajar

KATA KUNCI

Thread, Multithreading, Lightweight Process, pthread_create(), pthread_join(), pthread_mutex_t, Mutex, Kondisi Balapan, PID, TID, LWP, ps -T, htop -H, Memori Bersama, Pergantian Konteks, Satu-ke-Satu Model



LATAR BELAKANG TEORI

1. APA ITU THREAD DAN APA ITU MULTITHREADING?

Pada sistem operasi modern seperti Rocky Linux, program jarang berjalan sebagai rangkaian instruksi tunggal dan linier. Sebaliknya, program sering kali melakukan banyak tugas sekaligus—seperti mengunduh data sambil memperbarui antarmuka pengguna atau menangani banyak permintaan klien secara bersamaan. Kemampuan ini dimungkinkan melalui **thread** dan **multithreading**.

Thread adalah unit eksekusi terkecil dalam sebuah proses. Sementara **proses** mewakili program lengkap dengan ruang memorinya sendiri, **thread** berbagi ruang memori tersebut dengan thread lain dalam proses yang sama—tetapi mempertahankan stack, program counter, dan register set-nya sendiri.

Desain ini memungkinkan thread untuk bekerja secara bersamaan sambil berbagi sumber daya secara efisien.

Multithreading merujuk pada kemampuan suatu proses untuk membuat dan mengelola banyak thread yang dieksekusi secara bersamaan. Ini adalah konsep mendasar dalam pemrograman sistem modern, server web, aplikasi GUI, dan sistem real-time yang memungkinkan responsivitas, kinerja, dan pemanfaatan sumber daya yang efisien.

Bagian ini menjelaskan apa itu thread, bagaimana perbedaannya dengan proses, dan mengapa multithreading sangat penting dalam perangkat lunak di dunia nyata.

1.1. Thread sebagai proses ringan

Thread sering disebut sebagai **proses ringan** karena berperilaku seperti proses tetapi dengan overhead yang jauh lebih rendah.

Tidak seperti proses penuh yang membutuhkan ruang memori, deskriptor file, dan sumber daya kernel sendiri, sebuah thread memiliki fungsi-fungsi berikut:

- Berbagi kode, data, heap, dan file yang terbuka dengan thread lain dalam proses yang sama,
- Memiliki **stack**, **status register**, dan **ID thread (TID)** sendiri,
- Dapat dibuat dan dihancurkan jauh lebih cepat daripada sebuah proses.

Pemahaman inti: Membuat

thread baru biasanya **10–100 kali lebih cepat** daripada membuat proses baru karena tidak diperlukan duplikasi memori atau inisialisasi ulang sumber daya.

Di Linux, thread diimplementasikan sebagai **tugas** yang dikelola oleh kernel menggunakan mekanisme penjadwalan yang sama dengan proses—tetapi dikelompokkan di bawah **ID grup thread (TGID) umum**, yang sama dengan PID dari thread utama.



Anda dapat melihat utas menggunakan:

```
praktikumso@10:~$ ps -T -p [PID]
```

atau

```
praktikumso@10:~$ htop -H
```

1.2. Program Single-threaded vs Multithreaded

Program Berutas Tunggal

- Menjalankan satu instruksi dalam satu waktu.
 - Jika satu tugas terblokir (misalnya, menunggu I/O), seluruh program akan berhenti.
- Mudah ditulis dan di-debug—tetapi tidak efisien untuk aplikasi yang terikat I/O atau interaktif.

Contohnya: sebuah skrip shell dasar yang mengunduh sebuah file, lalu memprosesnya secara berurutan.

Program Multithread • Berisi

- **dua atau lebih thread** yang berjalan secara bersamaan dalam proses yang sama.
- Sementara satu thread menunggu I/O disk atau jaringan, thread lain dapat melanjutkan pemrosesan.
- Memungkinkan **responsivitas** (misalnya, GUI tetap dapat digunakan selama tugas latar belakang) dan **paralelisme** (pada CPU multi-core).

Contohnya: sebuah server web yang menangani 100 permintaan klien secara bersamaan menggunakan satu thread per permintaan.

Catatan: Multithreading menimbulkan kompleksitas—terutama ketika thread mengakses data bersama (lihat Bagian 4 tentang kondisi persaingan).

1.3. Thread vs Proses: perbedaan utama

Fitur	Proses	Benang
Ruang Memori	Ruang alamat virtual terpisah	Berbagi memori dengan yang lain thread dalam proses yang sama
Biaya Pembuatan Tinggi (salin tabel halaman, file deskriptor, dll.)		Rendah (berbagi sebagian besar sumber daya)
Komunikasi	Membutuhkan IPC (pipa, soket, memori bersama)	Akses langsung ke variabel bersama
Isolasi	Kegagalan dalam satu proses tidak memengaruhi proses lainnya.	Thread yang mengalami crash dapat menghentikan seluruh proses.
Pengalihan Konteks Biaya	Lebih tinggi (peralihan konteks memori penuh)	Lebih rendah (ruang memori yang sama)
Penggunaan Sumber Daya	CPU dan memori yang lebih besar	Ringan dan efisien



Aturan praktis:

Gunakan **beberapa proses** ketika Anda membutuhkan isolasi yang kuat (misalnya, keamanan, toleransi kesalahan).

Gunakan **banyak thread** ketika Anda membutuhkan performa dan akses data bersama dalam satu aplikasi.

1.4. Mengapa menggunakan Multithreading? Manfaat dalam aplikasi nyata

Multithreading bukan hanya konsep teoretis—ia mendukung sistem di dunia nyata: 1. **Responsivitas**

yang Lebih Baik • Dalam aplikasi GUI

(misalnya, editor teks, peramban), thread utama menangani input pengguna, sementara thread latar belakang melakukan penyimpanan, pengejaan, atau rendering. • Pengguna tidak pernah melihat antarmuka yang "macet".

2. **Berbagi Sumber Daya** •

Thread dalam proses yang sama berbagi kode, data, dan file yang terbuka—menghilangkan kebutuhan akan komunikasi antar-proses (IPC) yang kompleks.

3. **Ekonomi** •

Pembuatan, pengalihan, dan penghentian thread lebih murah daripada proses—penting untuk server berkinerja tinggi.

4. **Skalabilitas pada Sistem Multi-Core** • Pada

CPU modern dengan banyak core, thread dapat berjalan secara paralel—bukan hanya satu thread saja. secara bersamaan—mempercepat secara dramatis tugas-tugas yang membutuhkan banyak komputasi (misalnya, pengkodean video, simulasi ilmiah).

Contoh di Dunia Nyata:

• **Server Web** (Apache, Nginx): Satu thread per koneksi klien. • **Sistem Basis Data:**

Thread terpisah untuk penguraian kueri, I/O, dan pencatatan log. • **Pemutar Media:** Satu thread mendekode audio, thread lain merender video, dan thread ketiga menangani interaksi pengguna. kontrol.

Ringkasan:

Thread memungkinkan perangkat lunak yang efisien, responsif, dan skalabel dengan memungkinkan beberapa alur eksekusi dalam satu proses. Memahami perbedaan antara thread dan proses serta mengetahui kapan harus menggunakan multithreading sangat penting untuk pemrograman sistem, pengembangan server, dan optimasi kinerja di lingkungan Linux seperti Rocky Linux.

2. MODEL THREAD DALAM SISTEM OPERASI

Meskipun thread adalah unit eksekusi dasar dalam suatu proses, bagaimana thread **dikelola dan dijadwalkan** bergantung pada **model thread** yang digunakan oleh sistem operasi. Model yang berbeda menawarkan kompromi antara kinerja, portabilitas, dan keterlibatan kernel.

Memahami model-model ini membantu menjelaskan mengapa program multithreaded berperilaku berbeda di berbagai sistem dan mengapa Linux menggunakan pendekatan hibrida tertentu.



Bagian ini memperkenalkan tiga model thread utama: **Thread Tingkat Pengguna (ULT)**, **Thread Tingkat Kernel (KLT)**, dan **Model Hibrida (Banyak-ke-Banyak)**, dengan penekanan pada bagaimana Linux mengimplementasikan threading dalam praktiknya.

2.1. Thread Tingkat Pengguna (ULT)

A. Apa Itu Thread Tingkat Pengguna?

Thread Tingkat Pengguna (User-Level Threads/ULT) dikelola sepenuhnya oleh **pustaka thread ruang pengguna** (misalnya, GNU Portable Threads). Kernel **tidak menyadari** keberadaannya; kernel hanya melihat satu proses, meskipun proses tersebut berisi puluhan thread.

Cara Kerjanya

- Pembuatan thread, penjadwalan, dan peralihan konteks terjadi di ruang pengguna.
- Tidak diperlukan panggilan sistem untuk operasi thread → peralihan konteks sangat cepat.
- Satu thread kernel (proses) mendukung semua ULT.

Keunggulan •

- **Kinerja tinggi:** Operasi thread menghindari beban berlebih pada kernel.
- **Portabilitas:** Dapat dijalankan di sistem operasi apa pun, bahkan yang tidak memiliki dukungan thread bawaan.
- **Penjadwalan khusus:** Aplikasi dapat menentukan kebijakan penjadwalan sendiri.

Kekurangan • I/O

- **yang memblokir akan memblokir semua thread:** Jika satu ULT melakukan panggilan sistem yang memblokir (`read()`), Seluruh proses (dan semua thread-nya) terblokir.
- **Tidak ada paralelisme sejati:** Pada sistem multi-core, ULT tidak dapat berjalan secara bersamaan karena Kernel hanya menjadwalkan satu proses dalam satu waktu.

Contoh: Versi awal Java menggunakan ULT sebelum mengadopsi thread native.

2.2. Thread Tingkat Kernel (KLT)

A. Apa Itu Thread Tingkat Kernel?

Thread Tingkat Kernel (KLT) dikelola langsung oleh **kernel sistem operasi**. Setiap thread adalah entitas yang dapat dijadwalkan dan dikenal oleh kernel, sama seperti sebuah proses.

Cara Kerjanya

- Pembuatan, penjadwalan, dan sinkronisasi thread ditangani oleh kernel.
- Setiap thread memiliki **ID thread kernel (TID)** sendiri dan muncul di alat sistem seperti `'ps'` dan `'htop'`.
- Kernel dapat menjadwalkan beberapa thread dari proses yang sama pada inti CPU yang berbeda.



Keunggulan •

Konkurensi sejati: Beberapa thread dapat berjalan secara paralel pada sistem multi-core. • **Ketahanan:** Jika satu thread terblokir (misalnya, pada I/O), thread lain tetap melanjutkan eksekusi. • **Integrasi dengan layanan OS:** Thread mendapat manfaat dari penjadwalan tingkat kernel, sinyal, dan debugging.

Kekurangan •

Overhead lebih tinggi: Operasi thread memerlukan panggilan sistem dan keterlibatan kernel. • **Kurang portabel:** Membutuhkan dukungan OS untuk threading.

Linux menggunakan KLT sebagai dasar model threading-nya (melalui panggilan sistem 'clone()').

2.3. Model Hibrida dan Implementasi Linux

A. Model Hibrida (Banyak-ke-Banyak)

Model **Hibrida** menggabungkan ULT dan KLT:

- Beberapa thread tingkat pengguna dipetakan ke sejumlah thread kernel yang lebih kecil atau sama.
- Pustaka thread menangani penjadwalan ULT, sementara kernel menjadwalkan KLT.

Model ini bertujuan untuk mendapatkan yang terbaik dari kedua

- dunia: • Peralihan ruang pengguna yang cepat,
- Ditambah paralelisme sejati dan I/O non-blokir.

Namun, **penerapannya rumit** dan sebagian besar sudah tidak disukai lagi dalam sistem modern.

B. Bagaimana Linux Mengimplementasikan Threading: Model Satu-ke-Satu

Berbeda dengan buku teks lama, **Linux modern tidak menggunakan model hibrida**. Sebaliknya, ia menggunakan **model satu-ke-satu**:

- **Setiap thread pengguna = satu thread kernel.**
- Diimplementasikan melalui panggilan sistem 'clone()' , yang membuat tugas baru yang berbagi memori, deskriptor file, dan penanganan sinyal dengan induknya—tetapi memiliki tumpukan dan register sendiri.

Ini artinya:

- Setiap thread yang dibuat oleh 'pthread_create()' menjadi **entitas yang dapat dijadwalkan secara terpisah** di dalam kernel.
- Alat seperti 'ps -T' atau 'htop -H' menampilkan setiap thread sebagai baris terpisah dengan TID-nya masing-masing.



C. Verifikasi di Rocky Linux:

Kompilasi program pthread sederhana (lihat Bagian 5)

```
praktikumso@localhost:~$ gcc -o thread_demo.c -lpthread
```

Jalankan di latar belakang

```
praktikumso@localhost:~$ ./thread_demo &
```

Lihat utas

```
praktikumso@localhost:~$ ps -T -p $(pgrep thread_demo)
```

Output akan menampilkan beberapa TID di bawah PID yang sama—memastikan pemetaan satu-ke-satu.

D. Mengapa Hal Ini Penting bagi Siswa

Anda tidak perlu mengelola model thread secara manual. **Thread POSIX ('pthreads')** mengabstraksikan hal ini.

- Namun, memahami bahwa **setiap thread adalah entitas kernel yang sebenarnya** menjelaskan:
 - Mengapa 'htop' menampilkan banyak thread,
 - Mengapa menghentikan TID thread mengakhiri sebagian dari suatu proses,
 - Mengapa pembuatan thread memiliki overhead yang terukur.

Ringkasan :

Model thread mendefinisikan bagaimana thread dijadwalkan dan dikelola. Sementara ULT menawarkan kecepatan dan KLT menawarkan konkurensi sejati, **Linux menggunakan model tingkat kernel satu-ke-satu** melalui 'pthreads' dan 'clone()'. Ini memberi Anda kinerja dan paralelisme—sehingga ideal untuk server, aplikasi responsif, dan sistem waktu nyata.

3. PRAKTIK MULTITHREADING DI ROCKY LINUX

Meskipun konsep thread bersifat teoritis, implementasinya dalam sistem nyata bersifat konkret dan dapat diamati. Dalam sistem berbasis Linux seperti **Rocky Linux**, multithreading terutama didukung melalui API **POSIX Threads (pthreads)**—sebuah pustaka C standar untuk membuat dan mengelola thread. Bagian ini menjembatani teori dan praktik dengan memperkenalkan struktur dasar program pthread, mendemonstrasikan pembuatan thread, dan menunjukkan cara mengamati thread secara langsung di terminal.

Semua contoh dapat dikompilasi dan dijalankan di Rocky Linux menggunakan 'gcc' dan alat sistem standar—sehingga sangat ideal untuk sesi praktik di laboratorium.



3.1. POSIX Threads (pthreads): struktur dasar

A. Apa itu POSIX Threads (pthreads)?

POSIX Threads (pthreads) adalah API bahasa C terstandarisasi yang didefinisikan oleh standar IEEE POSIX 1003.1c. API ini menyediakan fungsi untuk:

- Membuat thread, •
- Mensinkronkan thread (menggunakan mutex, variabel kondisi, dll.), • Mengelola atribut thread.

Di Rocky Linux, pthread diimplementasikan sebagai bagian dari **glibc** (GNU C Library) dan menggunakan **model threading tingkat kernel satu-ke-satu** (setiap thread pengguna = satu tugas kernel).

B. Struktur Dasar Program pthread

Program C multithread minimal mencakup: 1. Sertakan

`'<pthread.h>'`

2. Definiskan fungsi thread (kode yang akan dijalankan oleh setiap thread)

3. Gunakan `'pthread_create()'` untuk membuat thread baru. 4.

Gunakan `'pthread_join()'` untuk menunggu thread selesai.

Aturan

Utama: Setiap thread memulai eksekusi pada fungsi yang sama tetapi dengan stack dan status register masing-masing.

Contoh kerangka:

```
1 #include <pthread.h>
2 #include <stdio.h>
3
4 //thread function
5 void* thread_function(void* arg) {
6     printf("Hello from a thread!\n");
7     return NULL;
8 }
9
10 int main() {
11     pthread_t tid;
12
13     //create thread
14     pthread_create(&tid, NULL, thread_function, NULL);
15
16     //Wait for thread to finish
17     pthread_join(tid, NULL);
18
19     printf("Thread complete.\n");
20     return 0;
21 }
```



Tips cepat: cara membuat file program C:

```
praktikumso@10:~$ vim thread_demo.c
```

Untuk melakukan kompilasi di Rocky Linux:

```
praktikumso@10:~$ gcc -o thread_demo thread_demo.c -lpthread
```

Catatan: Flag '-lpthread' menghubungkan pustaka pthread.

LATIHAN 1

Buat program C sederhana yang membuat satu thread menggunakan `pthread_create()`, kemudian kompilasi dan jalankan di Rocky Linux.

3.2. Membuat dan mengelola thread dengan 'pthread_create()'

A. Fungsi 'pthread_create()'

Sintaksis:

```
24 int pthread_create(
25     pthread_t *thread,
26     const pthread_attr_t *attr,
27     void *(*start_routine)(void*)
28     void *arg
29 );
```

B. Meneruskan Data ke Thread

Thread dapat menerima argumen melalui parameter '`void* arg`'. Ini memungkinkan penyesuaian per thread.

Contoh: Meneruskan ID berupa bilangan bulat:

```
32 void* worker(void* arg){
33     int id = *(int*)arg;
34     printf("Worker %d is running\n", id);
35     return NULL;
36 }
37
38 int main() {
39     pthread_t t1, t2;
40     int id1 = 1, id2 = 2;
41
42     pthread_create(&t1, NULL, worker, &id1);
43     pthread_create(&t2, NULL, worker, &id2);
44
45     pthread_join(t1, NULL);
46     pthread_join(t2, NULL);
47
48     return 0;
49 }
```



Perhatian: Hindari meneruskan variabel loop secara langsung di dalam loop—gunakan penyimpanan terpisah (misalnya, array ID) untuk mencegah kondisi persaingan (race condition) pada variabel itu sendiri.

Siklus Hidup Thread

- Sebuah thread dimulai ketika `'pthread_create()'` dipanggil.
- Thread berakhir ketika fungsi thread mengembalikan nilai atau `'pthread_exit()'` dipanggil.
- Thread utama harus menggabungkan semua thread anak menggunakan `'pthread_join()'` untuk menghindari penghentian prematur.

Praktik Terbaik: Selalu panggil `'pthread_join()'` kecuali Anda secara eksplisit melepaskan thread dengan `'pthread_detach()'`.

LATIHAN 2

Ubah program sebelumnya (`pthread_create()`) sehingga thread menerima ID berupa bilangan bulat sebagai argumen dan mencetaknya ke terminal.

3.3. Mengamati thread dalam sistem ('ps -T', 'htop')

Tidak seperti proses, thread berbagi PID yang sama—tetapi masing-masing memiliki ID Thread (TID) yang unik. Rocky Linux mengekspos TID ini melalui alat standar.

Menggunakan 'ps -T' untuk Melihat Thread

Opsi `'-T'` menampilkan semua thread dalam suatu proses:

```
praktikumso@10:~$ ./thread_demo &
praktikumso@10:~$ ps -T -p $(pgrep thread_demo)
```

Contoh output:

PID	SPID	TTY	WAKTU	CMD
1234	1234	poin/0	1234	0:00 thread_demo
1235	poin/0	1234	1236	0:00 thread_demo
poin/0				0:00 thread_demo

- **PID:** ID Proses (sama untuk semua thread)
- **SPID:** ID Thread (juga disebut LWP – Light Weight Process)

Kesimpulan: Semua thread memiliki PID yang sama tetapi memiliki SPID yang berbeda—mengkonfirmasi model kernel satu-ke-satu.

Menggunakan 'htop' untuk Memantau Thread

Jalankan `'htop'`, lalu tekan H (huruf H kapital) untuk beralih tampilan thread.



Dalam mode thread:

- Setiap thread ditampilkan sebagai baris terpisah.
- Thread utama biasanya tercantum pertama.
- Penggunaan CPU dan memori ditampilkan per thread.

Mengapa Ini Penting:

Mengamati thread membantu mendebug masalah kinerja (misalnya, satu thread mengonsumsi 100% CPU) atau memverifikasi bahwa multithreading berfungsi seperti yang diharapkan.

Ringkasan:

Multithreading di Rocky Linux praktis, dapat diamati, dan penting untuk pemrograman sistem modern. Dengan menggunakan `'pthread'`, Anda dapat membuat program konkuren yang memanfaatkan beberapa inti CPU. Alat seperti `'ps -T'` dan `'htop'` memungkinkan Anda memverifikasi perilaku thread langsung di terminal—penting untuk debugging dan penyetelan kinerja.

LATIHAN 3

Jalankan program multithread Anda di latar belakang, lalu gunakan `ps -T` untuk mengidentifikasi PID dan TID (ID Thread) dari setiap thread.

LATIHAN 4

Instal `htop` (jika belum terinstal), jalankan program multithread Anda, dan tekan H di `htop` untuk melihat semua thread sebagai entri terpisah.

4. MASALAH UMUM TERKAIT KONKURENSI DAN PRAKTIK AMAN

Multithreading memungkinkan kinerja dan responsivitas yang luar biasa—tetapi juga menghadirkan tantangan baru ketika beberapa thread mengakses data bersama secara simultan. Tanpa koordinasi yang tepat, thread dapat saling mengganggu, menyebabkan perilaku yang tidak dapat diprediksi, kerusakan data, atau ketidakstabilan sistem.

Bagian ini menjelaskan masalah konkurensi yang paling umum—kondisi persaingan (*race condition*)—dan memperkenalkan solusi paling sederhana dan paling banyak digunakan: eksklusivitas bersama (*mutual exclusion*) dengan mutex. Bagian ini juga menguraikan praktik terbaik untuk menghindari pola berbahaya seperti penghentian paksa (*forced termination*) dan kebuntuan (*deadlock*).

4.1. Kondisi persaingan (*race conditions*): ketika thread saling mengganggu

A. Apa itu Race Condition?

Kondisi **race** terjadi ketika dua atau lebih thread mengakses data bersama **secara bersamaan**, dan setidaknya salah satu di antaranya **memodifikasi** data tersebut, tanpa sinkronisasi yang tepat. Hasil akhirnya bergantung pada waktu atau **urutan** eksekusi—yang tidak dapat diprediksi.

Pemahaman Inti:



Kondisi race **tidak** berarti thread "berlomba" untuk menyelesaikan eksekusi—melainkan berarti kebenaran program **bergantung pada waktu**, yang tidak dapat diandalkan.

Contoh: Peningkatan Penghitung

Perhatikan kode C sederhana ini yang dieksekusi oleh dua thread:

```
51 int counter = 0;
52
53 void* increment(void* arg){
54     for(int i = 0; i < 100000; i++){
55         counter++;
56     }
57     return NULL;
58 }
```

Jika dua thread menjalankan 'increment()' secara bersamaan, nilai akhir 'counter' mungkin **kurang dari 200.000** karena operasi 'counter++' melibatkan: 1. Membaca nilai saat ini, 2. Menambahkan 1, 3. Menulis kembali.

Jika kedua thread membaca nilai yang sama sebelum salah satunya menulis, satu pembaruan akan hilang.

Dampak Nyata:

Dalam sistem perbankan, kondisi persaingan (race condition) dapat menyebabkan saldo yang tidak akurat. Dalam sistem tertanam (embedded systems), kondisi ini dapat memicu kegagalan keamanan.

B. Bagaimana Cara Mendeteksi Kondisi Balapan (Race Condition)?

- **Gejala:** Bug yang muncul sesekali, output yang tidak konsisten, crash saat beban kerja tinggi.
- **Alat:**

Gunakan 'helgrind' (bagian dari Valgrind) untuk mendeteksi data race:

```
praktikumso@10:~$ gcc -g -o race race.c -lpthread
```

```
praktikumso@10:~$ valgrind --tool=helgrind ./race
```

Catatan: Kondisi race condition seringkali hilang selama proses debugging—sehingga sulit untuk direproduksi.

LATIHAN 5

Tulis program C di mana **dua thread mengakses variabel global bersama** tanpa mutex, kemudian amati output yang tidak konsisten dengan menjalankan program beberapa kali.



4.2. Eksklusi bersama dasar dengan mutex ('pthread_mutex_t')

A. Apa Itu Mutex?

Mutex (singkatan dari mutual exclusion) adalah objek sinkronisasi yang memastikan hanya **satu thread** yang dapat mengakses bagian kode kritis pada satu waktu.

Dalam thread POSIX ('**threads**'), mutex dideklarasikan sebagai:

```
61 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

Menggunakan Mutex: Langkah

1. **Deklarasikan** mutex (global atau bersama).
2. **Kunci** sebelum memasuki bagian kritis.
3. **Buka kunci** setelah meninggalkannya.

Contoh (memperbaiki penghitung):

```
1 #include <pthread.h>
2 #include <stdio.h>
3
4 int counter = 0;
5 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
6
7 void* increment(void* arg){
8     for(int i = 0; i < 100000; i++){
9         pthread_mutex_lock(&lock);
10        counter++;
11        pthread_mutex_unlock(&lock);
12    }
13    return NULL;
14 }
15
```

Hasil: 'Penghitung' akhir akan selalu **200.000**.

Aturan Penting •

- **Selalu buka kunci** apa yang Anda kunci—jika tidak, thread lain akan **terblokir selamanya**.
- **Jangan pernah berasumsi urutan**: Thread dapat berjalan dalam urutan apa pun.
- **Jaga agar bagian kritis tetap kecil**: Menahan kunci terlalu lama mengurangi konkurensi.

Pengamatan di Rocky Linux: Saat

satu thread memegang mutex, thread lain yang memanggil '**pthread_mutex_lock()**' akan **terblokir** (tidur) hingga kunci dilepaskan—terlihat di 'htop' sebagai penurunan penggunaan CPU.

4.3. Praktik terbaik: menghindari pemutusan paksa dan kebuntuan

A. Hindari Penghentian Thread Secara Paksa



Jangan pernah menggunakan `'pthread_cancel()'` atau menghentikan thread secara tiba-tiba kecuali benar-benar diperlukan. Mengapa?

- **Sumber daya mungkin bocor:** File terbuka, memori, atau kunci tidak akan dibersihkan.
- **Mutex mungkin tetap terkunci:** Menyebabkan thread lain macet tanpa batas waktu.

Alternatif Aman: Gunakan **variabel flag** untuk memberi sinyal kepada thread agar keluar dengan baik:

```

1 #include <pthread.h>
2 #include <stdio.h>
3
4 volatile int stop_flag = 0;
5
6 void* worker(void* arg){
7     while (!stop_flag){
8         //DO work
9     }
10    // Clean up resource
11    return NULL
12 }
13
14 // To stop:
15 stop_flag = 1;
16 pthread_join(thread_id, NULL); //wait for clean exit
17

```

Catatan: pola ini digunakan pada server dan daemon di dunia nyata.

B. Hindari Kebuntuan

Deadlock terjadi ketika dua atau lebih thread **saling menunggu** untuk melepaskan sumber daya, menyebabkan semuanya macet selamanya.

Skenario Kebuntuan Klasik:

- Thread A mengunci mutex X, lalu mencoba mengunci mutex Y.
- Thread B mengunci mutex Y, lalu mencoba mengunci mutex X.
- Keduanya menunggu tanpa batas waktu.

Strategi Pencegahan:

1. **Pengurutan Penguncian:** Selalu peroleh mutex dalam **urutan global tetap** (misalnya, selalu kunci X) sebelum Y).
2. **Hindari Kunci Bersarang:** Jika memungkinkan, rancang kode agar hanya menggunakan satu mutex pada satu waktu.
3. **Gunakan Timeout** (lanjutan): `'pthread_mutex_timedlock()'` untuk menghindari penantian tak terbatas.

Aturan praktis:

Jika Anda memerlukan lebih dari satu mutex, **dokumentasikan urutan pengunciannya dengan jelas**.



Ringkasan Praktik Aman

Praktik	Mengapa Ini Penting
Lindungi data bersama dengan mutex. Mencegah kondisi persaingan (race condition).	
Jaga agar bagian-bagian kritis tetap pendek.	Memaksimalkan konkurensi
Jangan pernah mengakhiri thread secara tiba-tiba.	Menghindari kebocoran sumber daya dan kondisi yang rusak.
Gunakan urutan penguncian yang konsisten.	Mencegah kebuntuan
Pengujian di bawah beban	Kondisi balapan seringkali hanya muncul dengan konkurensi tinggi

Ringkasan:

Konkurensi itu ampuh tetapi membutuhkan disiplin. Dengan memahami kondisi persaingan (race condition) dan menggunakan mutex dengan benar, Anda dapat menulis program multithreaded yang cepat dan akurat. Ini keterampilan sangat penting untuk membangun aplikasi responsif, server web, dan sistem waktu nyata. Lingkungan Linux.



APLIKASI PRAKTIS

PRAKTIK

Tujuan: Mahasiswa membuat program C untuk menjalankan thread dan mensimulasikannya di terminal.
(Latihan belum ditampilkan; siswa menunggu instruksi selama sesi latihan.)

DEMO – STUDI KASUS

Situasi Sebuah

tim IoT kampus mengelola node sensor lingkungan yang berjalan di Rocky Linux. Setiap node menggunakan pengumpul data multithreaded untuk memproses pembacaan suhu. Selama pengujian beban, tim melaporkan pembacaan yang tidak konsisten—beberapa nilai duplikat sementara yang lain hilang sepenuhnya. Insinyur utama mencurigai adanya masalah konkurensi tetapi membutuhkan diagnosis segera. Kode yang bermasalah ada dalam dua versi: `counter_race.c` - tanpa sinkronisasi (rentan terhadap

kondisi race) berisi:

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #define ITER 100000
4 int counter = 0;
5 void* worker(void* arg) {
6     for (int i = 0; i < ITER; i++) counter++;
7     return NULL;
8 }
9 int main() {
10     pthread_t t1, t2;
11     pthread_create(&t1, NULL, worker, NULL);
12     pthread_create(&t2, NULL, worker, NULL);
13     pthread_join(t1, NULL);
14     pthread_join(t2, NULL);
15     printf("Final: %d\n", counter);
16     return 0;
17 }
18
```

`counter_mutex.c` - dengan perlindungan mutex (thread-safe) berisi:

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <time.h>
5
6 #define DEFAULT_ITERATIONS 100000
7
8 int counter = 0;
9 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
10 int num_threads = 2;
11 int iterations = DEFAULT_ITERATIONS;
12
13 void* worker(void* arg) {
14     // Tambahkan kode untuk menghitung counter dengan sinkronisasi
15     for (int i = 0; i < iterations; i++) {
16         pthread_mutex_lock(&lock);
17         counter++;
18         pthread_mutex_unlock(&lock);
19     }
20     return NULL;
21 }
22
```



```

24 int main(int argc, char* argv[]) {
25     // Tambahkan kode untuk:
26     // 1. Validasi argumen baris perintah
27     // 2. Inisialisasi barrier
28     // 3. Pengukuran waktu eksekusi per thread
29     // 4. Penanganan sumber daya
30
31     // Buat dan jalankan thread
32     pthread_t* threads = malloc(num_threads * sizeof(pthread_t));
33     if (!threads) {
34         perror("malloc failed");
35         return 1;
36     }
37
38     for (int i = 0; i < num_threads; i++) {
39         if (pthread_create(&threads[i], NULL, worker, NULL) != 0) {
40             perror("pthread_create failed");
41             // Cleanup: hentikan semua thread yang sudah dibuat
42             for (int j = 0; j < i; j++) {
43                 pthread_cancel(threads[j]);
44             }
45             free(threads);
46             return 1;
47         }
48     }
49
50     for (int i = 0; i < num_threads; i++) {
51         pthread_join(threads[i], NULL);
52     }
53
54     printf("\nFinal counter: %d\n", counter);
55     printf("Expected: %d\n", num_threads * iterations);
56
57     pthread_mutex_destroy(&lock);
58
59     free(threads);
60
61     return 0;
62 }

```

Catatan: File sumber C telah disiapkan sebelumnya di `~/demo6/`. Untuk membangun **memori otot** dan **kefasihan teknis**, Anda akan mengetik dan mengkompilasi skrip ini secara manual. Latihan ini dirancang untuk melatih Anda dalam penanganan sintaks yang presisi, membiasakan Anda dengan pustaka threading C, dan memastikan Anda dapat menavigasi lingkungan pengembangan Linux secara efektif di bawah tekanan. File sumber C telah disiapkan sebelumnya. Selama demo, Anda hanya akan **mengkompilasi, mengeksekusi, mengamati, memodifikasi, dan memverifikasi**, bukan menulis kode dari awal.

Tugas Anda adalah **mengkompilasi, menjalankan, mengamati, dan menjelaskan** perilaku kedua program tersebut hanya menggunakan perintah terminal, yang menunjukkan pemahaman Anda tentang thread, konkurensi, dan praktik yang aman.

Masalah yang Perlu Diidentifikasi

Sebagai insinyur keandalan sistem, Anda harus mendiagnosis masalah kritis ini dalam waktu 20 menit: • **Kondisi Balapan**

(Race Condition): Penghitung yang tidak sinkron menghasilkan hasil yang tidak konsisten (misalnya, 198.432 (bukan 200.000) karena adanya hazard baca-modifikasi-tulis.

• **Visibilitas Thread:** Alat sistem harus mengkonfirmasi beberapa thread yang berjalan di bawah satu sistem. proses



- **Validasi Sinkronisasi:** Versi yang dilindungi mutex harus secara konsisten menghasilkan output 200.000 di semua eksekusi.
- **Kompromi Kinerja:**

Implementasi mutex memperkenalkan overhead yang terukur yang harus diquantifikasi.

TUGAS DEMO (Mahasiswa Harus Melakukannya Secara Langsung di Terminal)

Tugas Anda adalah mendiagnosis masalah konkurensi dan memvalidasi solusinya melalui eksekusi terminal langsung. Anda akan berhasil jika:

1. Verifikasi Kondisi Balapan

- Kompilasi dan jalankan `counter_race.c` sebanyak 3 kali
- Dokumentasikan nilai penghitung yang tidak konsisten (misalnya, 198.432 ÷ 199.105 ÷ 197.888)
- Verifikasi jumlah thread menggunakan `ps -T -p [PID]` dan `htop` (tekan H)

2. Validasi Solusi Mutex

- Kompilasi dan jalankan `counter_mutex.c` sebanyak 3 kali
- Konfirmasikan output yang konsisten sebesar **200.000** di semua eksekusi
- Ukur dampak kinerja menggunakan `waktu ./counter_race` vs `waktu ./counter_mutex`

3. Modifikasi Desain Ubah

`counter_mutex.c` untuk mengimplementasikan sistem manajemen thread tingkat produksi yang:

- Penanganan Argumen Baris

Perintah: Menerima jumlah thread dinamis sebagai argumen pertama

(misalnya, `./counter_mutex 8`)

- Validasi Input: Verifikasi bahwa argumen adalah bilangan bulat antara 1-100, dan kembalikan pesan kesalahan yang sesuai jika tidak valid
- Implementasi Thread Barrier: Tambahkan barrier pthread untuk memastikan semua thread dapat dimulai secara bersamaan

Pemantauan Kinerja Per-Thread:

- Mencatat waktu mulai/berakhir untuk setiap thread
- Menghitung dan mencetak waktu eksekusi masing-masing thread
- Menampilkan waktu eksekusi thread rata-rata
- Verifikasi

Pembersihan Sumber Daya:

- Menghancurkan

- semua mutex dan barrier dengan benar
- Menambahkan langkah verifikasi untuk memastikan tidak ada kebocoran sumber daya

Optimasi Bagian Kritis:

- Mengurangi cakupan

- penguncian mutex hanya pada bagian kritis
- Menjelaskan mengapa optimasi ini meningkatkan kinerja



Persyaratan Verifikasi: •

- Mendemonstrasikan program bekerja dengan 4 thread (output = 400.000) •
- Menunjukkan penanganan kesalahan yang tepat dengan input yang tidak valid (misalnya, ./counter_mutex abc) • Menampilkan waktu eksekusi per thread dan rata-ratanya • Memastikan semua sumber daya dibersihkan dengan benar

4. Penjelasan Teknis •

- Jelaskan mengapa kondisi race terjadi di counter_race.c • Jelaskan bagaimana mutex mencegah kerusakan data • Jelaskan pertukaran antara kinerja dan kebenaran yang diamati

Aturan Demo

1. Tidak Ada Sumber

Eksternal Mahasiswa tidak boleh membuka PDF modul, catatan, atau internet selama demo.

2. Berkas yang Sudah Disiapkan

Hanya file counter_race.c dan counter_mutex.c yang tersedia di ~/demo6/. Tidak ada penulisan kode—hanya kompilasi, eksekusi, dan pengamatan.

3. Persyaratan Verifikasi Semua

- klaim harus didukung oleh output terminal langsung: • Nilai penghitung yang tidak konsisten vs. konsisten
- Daftar thread melalui ps -T • Perbandingan waktu melalui time

4. Fokus Penilaian

- Penilai akan mengevaluasi:
 - Penggunaan alat sistem yang tepat untuk melihat thread
 - Pemahaman tentang kondisi race vs. perlindungan mutex • Kemampuan untuk mengartikulasikan pertimbangan sinkronisasi

5. Respons Interaktif

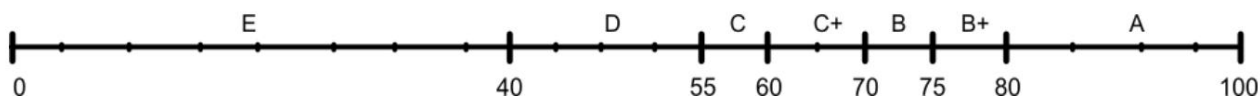
- Mahasiswa harus menjawab pertanyaan seperti: • "Tunjukkan TID untuk proses ini" • "Jelaskan mengapa kondisi race terjadi" • "Apa yang terjadi jika kita lupa memanggil pthread_mutex_unlock()?"



RUBRIK PENILAIAN

Aspek Penilaian	Poin (Total 100%)
LATIHAN	Total 5%
Latihan 1	1%
Latihan 2	1%
Latihan 3	1%
Latihan 4	1%
Latihan 5	1%
PRAKTIK	Total 20%
Latihan 1	10%
Latihan 2	10%
DEMO	Total 75%
Penyelesaian Studi Kasus	35%
Memahami Kondisi Balapan & Diagnosis	10%
Memahami Mutex & Sinkronisasi	10%
Keterampilan Modifikasi Desain	10%
Analisis Pertukaran Kinerja-Keakuratan	10%

SKALA PENILAIAN



A = (81 - 100) ÿ Sepuh

B+ = (75 - 80) ÿ Sangat Baik

B = (70 - 74) ÿ Baik

C+ = (60 - 69) ÿ Cukup Baik

C = (55 - 59) ÿ Adil

D = (41 - 54) ÿ Miskin

E = (0 - 40) ÿ Bro sungguh...

